
Statistics for AI/ML-II

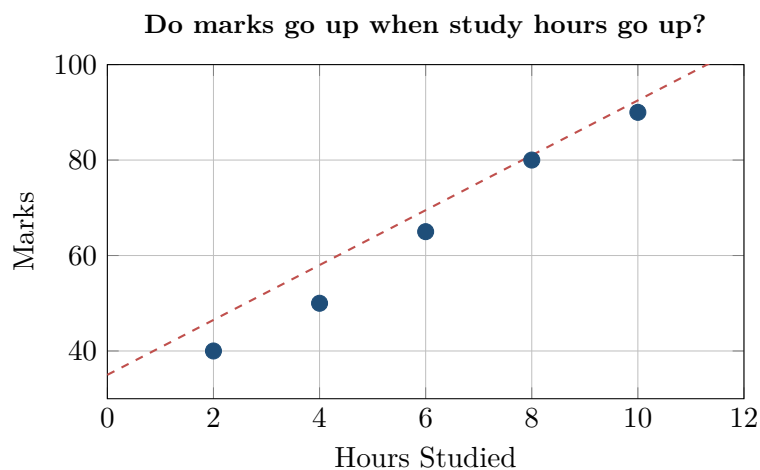
Chapter 1

Correlation & Covariance

1.1 Why We Need These Tools

Consider a classroom where we record how long each student studied and the marks they obtained:

Hours Studied	Marks
2	40
4	50
6	65
8	80
10	90



The question: Do two variables move together, and *how strongly*?

We need a numerical tool for this, leading to two related concepts:

Covariance $\longrightarrow$ **Correlation**

1.2 Covariance: The Intuition

1.2.1 Real-Life Analogy: Two Friends Walking

Imagine two friends hiking. We observe how their altitudes change over time.

Case 1: Up Together

↑ Friend A

↑ Friend B

Cov > 0

Case 2: Opposite

↑ Friend A

↓ Friend B

Cov < 0

Case 3: No Pattern

↑↓↑↓

↓↑↓↑

Cov ≈ 0

1.3 Mathematical Definition

CovarianceFor a **population** of N observations:

$$\text{Cov}(X, Y) = \frac{1}{N} \sum_{i=1}^N (X_i - \mu_X)(Y_i - \mu_Y)$$

For a **sample** of n observations:

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})$$

where $\bar{X}, \bar{Y}$ are sample means and μ_X, μ_Y are population means.

1.4 Computing Covariance

Worked Example

Given:

$$X = [1, 2, 3, 4, 5], \quad Y = [2, 4, 6, 8, 10]$$

Step 1: Compute the means.

$$\bar{X} = \frac{1+2+3+4+5}{5} = 3, \quad \bar{Y} = \frac{2+4+6+8+10}{5} = 6$$

Step 2: Tabulate the deviations and their products.

X_i	Y_i	$X_i - \bar{X}$	$Y_i - \bar{Y}$	$(X_i - \bar{X})(Y_i - \bar{Y})$
1	2	-2	-4	8
2	4	-1	-2	2
3	6	0	0	0
4	8	1	2	2
5	10	2	4	8
Sum:				20

Step 3: Apply the sample formula.

$$\text{Cov}(X, Y) = \frac{20}{5-1} = 5$$

Interpretation: Covariance is positive, so X and Y move together.

1.5 Covariance in Python

```

1  import numpy as np
2
3  x = np.array([1, 2, 3, 4, 5])
4  y = np.array([2, 4, 6, 8, 10])
5
6  cov_matrix = np.cov(x, y)
7  print(cov_matrix)

```

Listing 1.1: Computing covariance with NumPy

Output:

```

[[ 2.5  5. ]
 [ 5. 10. ]]

```

How to Read the Covariance Matrix

The covariance matrix is symmetric:

$$\begin{bmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Var}(Y) \end{bmatrix} = \begin{bmatrix} 2.5 & 5.0 \\ 5.0 & 10.0 \end{bmatrix}$$

- **Diagonal:** variance of each variable.
- **Off-diagonal:** covariance between variables.
- **Always symmetric:** $\text{Cov}(X, Y) = \text{Cov}(Y, X)$.

1.6 The Problem With Covariance: Units Matter

The Unit Trap

Suppose we measure the same heights and weights in two unit systems:

Dataset	Covariance
Heights in metres, weights in kg	≈ 2
Heights in centimetres, weights in grams	$\approx 200,000$

Same relationship – wildly different numbers.

The magnitude of covariance is hostage to units. We need a *scale-free* measure. That measure is **correlation**.

1.7 The Pearson Correlation Coefficient

Pearson Correlation Coefficient

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

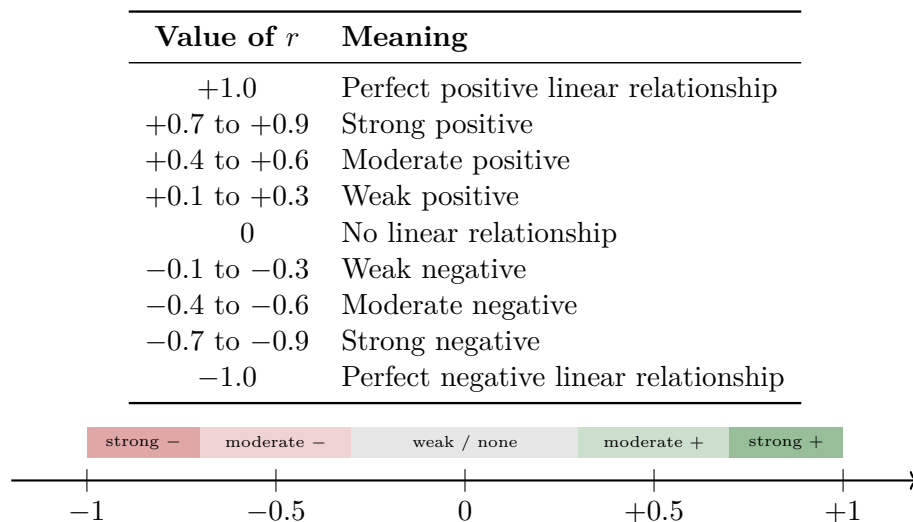
That is: *covariance divided by the product of standard deviations.*

1.7.1 Why Dividing by $\sigma_X \sigma_Y$ Works

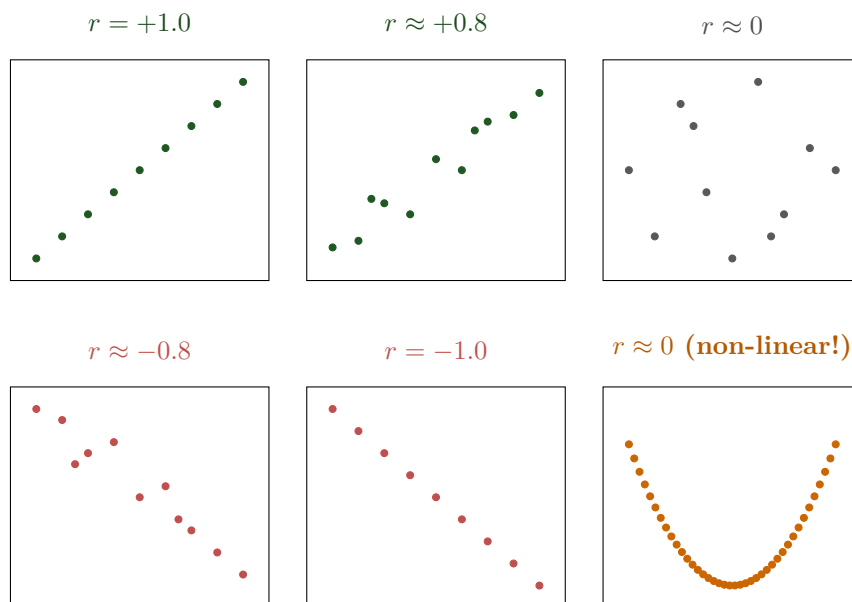
Standard deviations carry the same units as X and Y . Dividing cancels those units and rescales the result. The output is a **pure number**, always satisfying:

$$-1 \leq r \leq 1$$

1.7.2 Interpretation Scale



1.8 Visual: What Different Correlations Look Like



Important

The bottom-right plot has **no linear correlation** ($r \approx 0$) yet a clear, deterministic relationship $Y = 0.4X^2 + 1$. Pearson correlation only captures *linear* association.

1.9 Manual Correlation Example

Same X, Y as Before

We already found:

$$\text{Cov}(X, Y) = 5$$

Now compute the standard deviations:

$$\sigma_X = \sqrt{\frac{\sum (X_i - \bar{X})^2}{n-1}} = \sqrt{\frac{10}{4}} = \sqrt{2.5} \approx 1.581$$

$$\sigma_Y = \sqrt{\frac{\sum (Y_i - \bar{Y})^2}{n-1}} = \sqrt{\frac{40}{4}} = \sqrt{10} \approx 3.162$$

Therefore:

$$r = \frac{5}{1.581 \times 3.162} = \frac{5}{5.0} \approx 1.0$$

A perfect positive correlation – exactly what we'd expect because $Y = 2X$.

1.10 Correlation in Python

```

1  import numpy as np
2
3  x = np.array([1, 2, 3, 4, 5])
4  y = np.array([2, 4, 6, 8, 10])
5
6  corr = np.corrcoef(x, y)
7  print(corr)
8  # [[1.  1.]
9  #   [1.  1.]]

```

Listing 1.2: NumPy: `np.corrcoef`

```

1  import pandas as pd
2
3  df = pd.DataFrame({
4      "Hours": [1, 2, 3, 4, 5],
5      "Marks": [2, 4, 6, 8, 10]
6  })
7
8  print(df.corr())
9  #           Hours  Marks
10 # Hours      1.0    1.0
11 # Marks      1.0    1.0

```

Listing 1.3: Pandas: `df.corr()`

1.11 The Correlation Matrix

Real datasets have many features. We want the correlation between *every pair* of features.

Employee Dataset			
Features: {Age, Salary, Experience}			
	Age	Salary	Experience
Age	1.00	0.82	0.88
Salary	0.82	1.00	0.91
Experience	0.88	0.91	1.00

Properties of a Correlation Matrix

- It is **square** ($p \times p$ for p features).
- It is **symmetric**: $r_{ij} = r_{ji}$.
- The diagonal is always **1** (a variable correlates perfectly with itself).
- Every entry lies in $[-1, +1]$.

1.12 Visualizing with a Heatmap

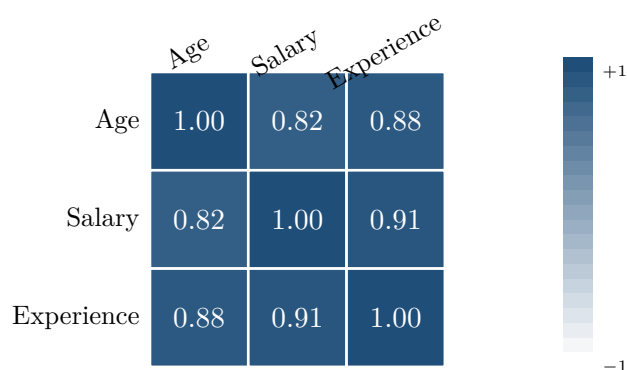
```

1  import seaborn as sns
2  import matplotlib.pyplot as plt
3
4  corr = df.corr()
5
6  sns.heatmap(
7  corr,
8  annot=True,
9  cmap="coolwarm",
10 vmin=-1, vmax=1
11 )
12 plt.title("Correlation Matrix")
13 plt.show()

```

Listing 1.4: Seaborn heatmap

A simulated heatmap looks like:



```

1  import seaborn as sns
2  import matplotlib.pyplot as plt
3
4  tips = sns.load_dataset("tips")
5  corr = tips.corr(numeric_only=True)
6
7  sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
8  plt.title("Correlation Matrix -- Tips Dataset")
9  plt.show()

```

Listing 1.5: Heatmap on the Tips dataset

1.13 Covariance vs Correlation

Property	Covariance	Correlation
Measures relationship	✓	✓
Indicates direction	✓	✓
Indicates strength	×	✓
Unit-dependent	✓	×
Bounded / fixed range	×	✓ $([-1, +1])$
Easy to interpret	×	✓
Used in PCA mathematics	✓	sometimes
Used in feature inspection	sometimes	✓

Quick Mnemonic

Covariance answers “*which direction?*” **Correlation** answers “*which direction **and** how strong?*”

1.14 Usecase with Machine Learning

1.14.1 Feature Selection

Suppose you are predicting student marks:

Feature	Correlation with Target
Hours Studied	+0.90
Attendance	+0.82
Sleep Hours	+0.35
Shoe Size	+0.01

- **Keep:** Hours Studied, Attendance (strong predictors)
- **Consider:** Sleep Hours (weak but possibly useful)
- **Drop:** Shoe Size (essentially irrelevant)

1.14.2 Multicollinearity Detection

When two input features are themselves highly correlated, linear models become unstable – coefficients flip signs, standard errors balloon, and interpretation breaks down.

The Income Trap

	Salary	Monthly Income	Annual Income
Salary	1.00	0.99	0.99
Monthly Income	0.99	1.00	1.00
Annual Income	0.99	1.00	1.00

These three columns essentially carry the same information. **Keep one**, drop the others.

```

1  import pandas as pd
2
3  corr = df.corr().abs()
4  # pull pairs with |r| > 0.9 (excluding the diagonal)
5  upper = corr.where(
6  pd.DataFrame(
7  [[i < j for j in range(len(corr))] for i in range(len(corr))],
8  index=corr.index, columns=corr.columns
9  )
10 )
11 high = upper.stack().loc[lamba s: s > 0.9]
12 print(high)

```

Listing 1.6: Detecting multicollinearity

1.14.3 Linear Regression

Simple linear regression fits:

$$Y = \beta_0 + \beta_1 X$$

And it turns out that:

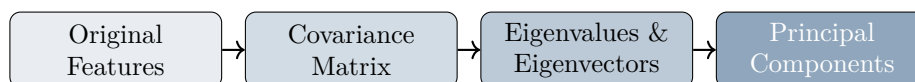
$$\beta_1 = \frac{\text{Cov}(X, Y)}{\text{Var}(X)} = r \cdot \frac{\sigma_Y}{\sigma_X}$$

So the slope is *literally derived* from covariance. Higher correlation $\Rightarrow$ tighter fit $\Rightarrow$ better predictive power (for linear relationships).

1.14.4 Principal Component Analysis (PCA)

PCA begins by computing the covariance matrix of the (centered) features:

Covariance matrix $\longrightarrow$ Eigen-decomposition $\longrightarrow$ Principal components



If two features (say, height in cm and weight in kg) are highly correlated, PCA collapses them into a single combined axis, reducing dimensionality without losing much information.

1.14.5 Recommendation Systems

Item-based collaborative filtering uses correlation between users' rating patterns:

Movie Recommender

User	<i>Interstellar</i>	<i>Inception</i>	<i>The Matrix</i>	<i>Notebook</i>
Alice	5	5	4	1
Bob	4	5	5	2
Carol	5	4	5	1
Dave	2	1	2	5

Computing pairwise correlations across users (or items) reveals which movies cluster together. “Users who liked *Interstellar* & *Inception*” will get *The Matrix* recommended next.

1.14.6 End-to-End ML Example: Iris Dataset

```

1  import pandas as pd
2  import seaborn as sns
3  import matplotlib.pyplot as plt
4
5  # Load dataset
6  iris = sns.load_dataset("iris")
7
8  # Correlation matrix (numeric columns only)
9  corr = iris.drop(columns=["species"]).corr()
10 print(corr)
11
12 # Heatmap
13 sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
14 plt.title("Iris -- Feature Correlation Matrix")
15 plt.show()

```

Listing 1.7: Correlation analysis on Iris

What to look for:

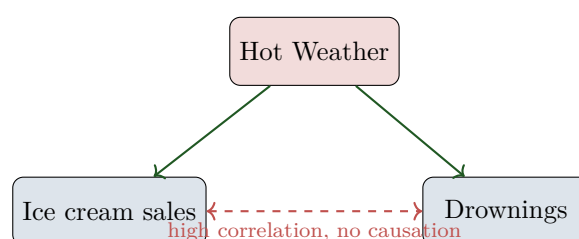
- `petal_length` and `petal_width` typically show $r \approx 0.96$ – almost duplicated information.
- `sepal_width` is weakly correlated with the others – it carries unique signal.
- *Decision*: keep `petal_length` + `sepal_width`, or apply PCA before training.

1.15 Common Pitfalls (issues)

1.15.1 Pitfall 1 – Correlation $\neq$ Causation

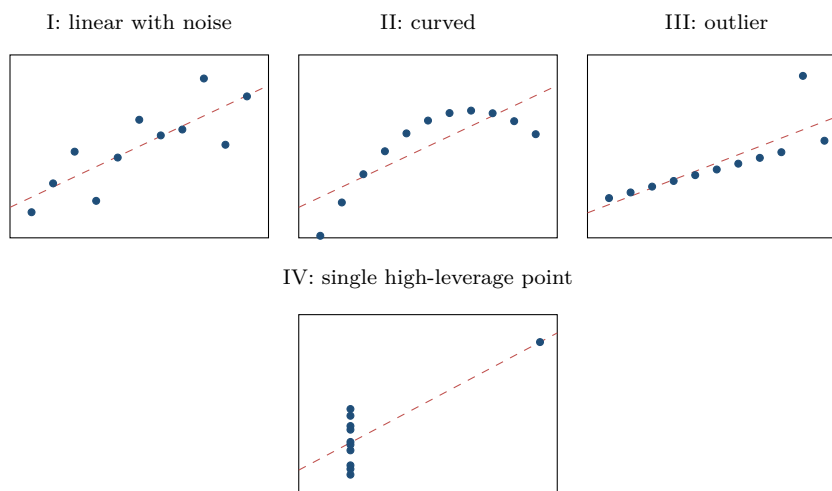
The Classic Trap

Ice-cream sales and drowning deaths are strongly correlated. **Ice cream does not cause drowning.** A *third variable* – summer heat – drives both.



1.15.2 Pitfall 2 – Anscombe’s Quartet

Four very different datasets can share the *exact same* mean, variance, regression line, **and correlation** ($r \approx 0.816$). Always plot your data.



Takeaway: a single correlation number can hide curves, outliers, and structural issues. *Always visualize.*

1.15.3 Pitfall 3 – Non-linear Relationships

Pearson’s r only detects **linear** association. For monotonic but non-linear relationships, use **Spearman’s rank correlation** instead.

Method	Detects	When to use
Pearson	Linear relationships	Variables are roughly linear and continuous
Spearman	Monotonic relationships (any shape, as long as it’s consistently up or down)	Ordinal data, or non-linear monotonic data
Kendall	Concordance of pairs	Small samples, many ties


```

1 df.corr(method="pearson") # default
2 df.corr(method="spearman") # rank-based, monotonic
3 df.corr(method="kendall") # tau, robust to ties

```

Listing 1.8: Using different correlation methods in Pandas

1.15.4 Pitfall 4 – Outliers Dominate

A single extreme point can flip a correlation from positive to negative. Always inspect with a scatter plot before trusting r .

1.16 Key Takeaways

Covariance

- Measures whether two variables move together.
- **Sign** indicates direction; **magnitude** is unit-dependent.
- Foundation for variance structure (PCA).

Correlation

- Standardized covariance: $r = \text{Cov}(X, Y) / (\sigma_X \sigma_Y)$.
- Always in $[-1, +1]$; unit-free.
- Indicates *both* direction and *strength*.
- Pearson = linear; Spearman = monotonic.

Practical Rule of Thumb

Use **covariance** when studying variance structure (e.g. PCA mathematics, multivariate Gaussians). Use **correlation** when communicating relationships between features, because it is unit-free and instantly interpretable.

Chapter 2

Sampling

2.1 Population vs. Sample

The **population** is everything you want to draw conclusions about; the **sample** is what you actually observe.

2.2 Sampling Methods

- **Simple random sampling:** each individual has equal probability.
- **Stratified sampling:** split population into strata (e.g., classes) and sample from each. Crucial for imbalanced data.
- **Cluster sampling:** sample whole groups (clusters), then everyone within.
- **Systematic sampling:** every k -th element.
- **Convenience / non-probability sampling:** biased, but common in practice.

2.3 Sampling Bias

Sampling bias happens when the data we collect does not properly represent the whole population. As a result, the conclusions we make and the machine learning models we build may be inaccurate.

Common types of sampling bias include:

- **Selection Bias:** Some groups are more likely to be chosen than others.
- **Survivorship Bias:** We only look at successful cases and ignore failures.
- **Response Bias:** People may not answer surveys truthfully.
- **Coverage Bias:** Some parts of the population are left out completely.

AI/ML Application

In machine learning, we want the training, validation, and test datasets to have a similar distribution.

For classification problems, we often use **stratified sampling** so that each class appears in the same proportion in all datasets.

“**Cross-validation** repeatedly splits the available data into training and testing sets to get a more reliable estimate of model performance.”

```
1      import numpy as np
2      from sklearn.model_selection import train_test_split
3
4      # 90 students passed (0), 10 students failed (1)
5      y = np.array([0] * 90 + [1] * 10)
6
7      # Without stratification
8      y_train, y_test = train_test_split(y, test_size=0.2, random_state=42)
9
10     print("Without Stratification")
11     print("Train failure percentage:", round(y_train.mean() * 100, 2), "%")
12     print("Test failure percentage :", round(y_test.mean() * 100, 2), "%")
13
14     # With stratification
15     y_train, y_test = train_test_split(
16     y,
17     test_size=0.2,
18     stratify=y,
19     random_state=42
20     )
21
22     print("\nWith Stratification")
23     print("Train failure percentage:", round(y_train.mean() * 100, 2), "%")
24     print("Test failure percentage :", round(y_test.mean() * 100, 2), "%")
```

Listing 2.1: Stratified train/test split preserves class proportions

Chapter 3

Parameter Estimation

3.1 Point Estimation

Given data D , estimate a parameter θ with a single value $\hat{\theta}(D)$.

3.1.1 Properties of Estimators

- **Bias:** $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$. Unbiased if zero.
- **Variance:** $\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$.
- **Mean Squared Error:** $\text{MSE}(\hat{\theta}) = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2$ — the bias–variance decomposition.
- **Consistency:** $\hat{\theta}_n \rightarrow \theta$ as $n \rightarrow \infty$.
- **Efficiency:** lowest possible variance (Cramér–Rao bound).

AI/ML Application

The bias–variance decomposition isn’t just abstract – it is *the* reason regularization helps. Adding bias (e.g., shrinking weights toward zero) can lower variance enough to reduce MSE overall.

```
1      import numpy as np
2
3      true_mu = 5.0
4      n_per_sample = 20
5      n_simulations = 5000
6
7      estimates = []
8      for _ in range(n_simulations):
9          sample = np.random.normal(true_mu, 2.0, n_per_sample)
10         estimates.append(sample.mean())           # sample mean as estimator
11
12         estimates = np.array(estimates)
13         bias      = estimates.mean() - true_mu
14         variance  = estimates.var()
15         mse       = ((estimates - true_mu) ** 2).mean()
16
17         print(f"Bias      = {bias:.4f}")
18         print(f"Variance  = {variance:.4f}")
19         print(f"MSE      = {mse:.4f} (should equal Bias^2 + Variance)")
20         print(f"Bias^2+V  = {bias**2 + variance:.4f}")
```

Listing 3.1: Empirical bias–variance of an estimator

3.2 Maximum Likelihood Estimation (MLE)

Given data $D = \{x_1, \dots, x_n\}$ from $p(x | \theta)$, the likelihood is

$$L(\theta) = \prod_{i=1}^n p(x_i | \theta), \quad \ell(\theta) = \log L(\theta) = \sum_{i=1}^n \log p(x_i | \theta).$$

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \ell(\theta).$$

Intuition

Suppose a teacher wants to estimate a student’s ability based on an exam score of 95 out of 100.

“**MLE (Maximum Likelihood Estimation)** asks:

“Which ability level is most likely to produce a score of 95?”

In machine learning, we similarly look for the model parameters that make the observed data most likely.

Cross-Entropy measures how surprised the model is by the correct answer.

If the model predicts:

$$P(\text{Correct Answer}) = 0.95$$

then the model is not surprised, so the cross-entropy loss is small.

But if the model predicts:

$$P(\text{Correct Answer}) = 0.05$$

then the model is very surprised, so the cross-entropy loss is large.

Therefore:

- MLE tries to make the observed data as likely as possible.
- Cross-Entropy tries to reduce the model’s surprise about the observed data.

These are two ways of expressing the same goal, which is why maximizing likelihood is equivalent to minimizing cross-entropy. “

AI/ML Application

Cross-entropy loss is MLE under a categorical model.

MSE is MLE under a Gaussian noise model.

L2 regularization is MAP with a Gaussian prior on weights.

L1 regularization is MAP with a Laplace prior on weights.

```

1      import numpy as np
2      from scipy.optimize import minimize
3      from scipy.stats import norm
4
5      # Generate data
6      np.random.seed(0)
7      data = np.random.normal(loc=5.0, scale=2.0, size=200)
8

```



```

9      # --- Closed-form MLE for Normal ---
10     mu_hat      = data.mean()
11     sigma_hat   = data.std(ddof=0)          # MLE uses N, not N-1
12     print(f"Closed-form  mu = {mu_hat:.3f},  sigma = {sigma_hat:.3f}")
13
14     # --- Numerical MLE by minimizing negative log-likelihood ---
15     def neg_log_lik(params, x):
16         mu, log_sigma = params
17         sigma = np.exp(log_sigma)          # ensure positivity
18         return -np.sum(norm.logpdf(x, loc=mu, scale=sigma))
19
20     res = minimize(neg_log_lik, x0=[0.0, 0.0], args=(data,))
21     mu_num, sigma_num = res.x[0], np.exp(res.x[1])
22     print(f"Numerical    mu = {mu_num:.3f},  sigma = {sigma_num:.3f}")

```

Listing 3.2: MLE for a Gaussian: closed form vs numerical

```

1      import numpy as np
2      from sklearn.linear_model import LogisticRegression
3      from sklearn.datasets import make_classification
4      from scipy.special import expit      # logistic sigmoid
5
6      X, y = make_classification(n_samples=500, n_features=3, random_state
7                                =0)
8
9      clf = LogisticRegression(penalty=None, solver="lbfgs").fit(X, y)
10
11     # Hand-computed log-likelihood at the fitted weights
12     w, b = clf.coef_.ravel(), clf.intercept_[0]
13     p = expit(X @ w + b)
14     log_lik = np.sum(y * np.log(p) + (1 - y) * np.log(1 - p))
15     print(f"Log-likelihood at MLE solution: {log_lik:.2f}")
16     print(f"(equivalently, -BCE * N = -{-log_lik:.2f})")

```

Listing 3.3: Logistic regression = MLE under a Bernoulli model

3.3 Interval Estimation: Confidence Intervals

Suppose a school has 10,000 students, and you want to know their average height.

Measuring every student would take a lot of time, so you randomly select 100 students and calculate their average height. Since different samples give slightly different averages, instead of reporting a single number, you report a range of plausible values.

This range is called a **confidence interval (CI)**.

For example, instead of saying:

[Average Height = 170 cm]

you might say:

[168,172] cm]

This means the true average height is likely to be somewhere in this range.

For a Gaussian sample mean with known σ :

$$\bar{x} \pm z_{\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}.$$

For unknown σ , replace z with the Student- t critical value and σ with s :

$$\bar{x} \pm t_{\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}.$$

Intuition

Imagine repeatedly selecting 100 students from the same school and computing a confidence interval each time.

“Some intervals will contain the true average height and some will miss it.

A **95% confidence interval** means that if we repeat this process many times, about 95 out of 100 intervals will contain the true value.

It does *not* mean:

“There is a 95% chance that the true value is inside this particular interval.”

Instead, it means:

“The method used to create the interval is successful about 95% of the time.” “

AI/ML Application

In machine learning, a single accuracy score can be misleading.

“For example, a model may achieve 90% accuracy on one test set and 88% on another.

A confidence interval shows the uncertainty around the metric.

For example:

$$\text{Accuracy} = 89\% \pm 2\%$$

This tells us that the model’s true performance is likely to be close to this range rather than exactly 89%.

Bootstrapping is a simple and widely used technique for estimating such confidence intervals. “

```

1      import numpy as np
2      from scipy import stats
3
4      data = np.random.normal(loc=10, scale=2, size=30)
5      n    = len(data)
6      mean = data.mean()
7      sem  = stats.sem(data)                                # s / sqrt(n)
8      ci   = stats.t.interval(0.95, df=n-1, loc=mean, scale=sem)
9      print(f"95% CI for the mean: [{ci[0]:.2f}, {ci[1]:.2f}]")

```

Listing 3.4: t-based CI for a population mean

Chapter 4

Hypothesis Testing

4.1 The Framework

- **Null hypothesis H_0 :** the status quo (no effect).
- **Alternative H_1 :** what you want to detect.
- **Test statistic T :** a function of the data.
- **p-value:** probability of seeing a result at least as extreme as observed if H_0 were true.
- **Significance level α :** threshold (commonly 0.05).
- Reject H_0 if p-value $< \alpha$.

Intuition

The p-value answers: “If the null world were true, how often would we see data this weird?” A tiny p-value means our data is wildly inconsistent with H_0 , so we abandon it. It does *not* mean H_1 is probably true – that’s a Bayesian claim.

4.2 The Two Types of Errors

	H_0 true	H_0 false
Reject H_0	Type I error (α)	Correct (power = $1 - \beta$)
Fail to reject H_0	Correct ($1 - \alpha$)	Type II error (β)

- **Type I (False positive):** we claim an effect when there is none.
- **Type II (False negative):** we miss a real effect.
- **Power = $1 - \beta$:** probability of correctly detecting a true effect. Driven up by larger samples, larger effect sizes, and lower noise.

4.3 Step-by-Step Recipe

1. State H_0 and H_1 .
2. Choose α (e.g., 0.05).
3. Pick the appropriate test based on data type and assumptions.

4. Compute the test statistic and p-value.
5. Compare: reject H_0 if $p < \alpha$; otherwise fail to reject.
6. Report the *effect size* – statistical significance without effect size is misleading.

4.4 Choosing the Right Test

Goal	Setting	Test
Compare mean to a value	1 continuous sample, large n , known σ	z-test
Compare mean to a value	1 continuous sample, unknown σ	one-sample t-test
Compare two means	Two independent samples	two-sample t-test
Compare two means	Paired/matched samples	paired t-test
Compare ≥ 3 means	Multiple groups	ANOVA (F-test)
Test a proportion	Binary outcomes	z-test for proportions
Compare two proportions	Two independent binary outcomes	chi-squared / z-test
Goodness of fit	Categorical distribution	chi-squared
Independence of categoricals	Contingency table	chi-squared
Non-parametric comparison	Two samples, non-normal	Mann–Whitney U
Paired non-parametric	Two paired non-normal samples	Wilcoxon signed-rank
Compare two distributions	Continuous, any shape	Kolmogorov–Smirnov
Test linear correlation	Two continuous variables	Pearson correlation test

4.5 One-Sample t-Test

Question: Does the mean differ from a known value μ_0 ?

$$H_0 : \mu = \mu_0, \quad H_1 : \mu \neq \mu_0, \quad t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}.$$

Scenario

A model documentation claims average inference latency is 100 ms. You measure 25 requests. Is the true mean actually 100 ms?

```

1      import numpy as np
2      from scipy import stats
3
4      # Observed latencies (ms)
5      np.random.seed(1)
6      latencies = np.random.normal(loc=105, scale=8, size=25)
7
8      t_stat, p_val = stats.ttest_1samp(latencies, popmean=100)
9      print(f"sample mean = {latencies.mean():.2f} ms")
10     print(f"t = {t_stat:.3f}, p = {p_val:.4f}")
11
12     alpha = 0.05
13     if p_val < alpha:
14         print("Reject H0: the true mean latency is NOT 100 ms.")
15     else:
16         print("Fail to reject H0: no evidence the mean differs from 100 ms.")

```

Listing 4.1: One-sample t-test

4.6 Two-Sample t-Test (Independent)

Question: Do two independent groups have the same mean?

$$H_0 : \mu_A = \mu_B, \quad t = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{s_A^2/n_A + s_B^2/n_B}}.$$

A/B Test on Conversion Time

Group A uses the old checkout flow; Group B uses the new one. Compare average time-to-purchase.

```

1      import numpy as np
2      from scipy import stats
3
4      np.random.seed(0)
5      group_A = np.random.normal(loc=120, scale=15, size=80) # old flow
6      group_B = np.random.normal(loc=112, scale=18, size=85) # new flow
7
8      # Welch's t-test (does NOT assume equal variances)
9      t_stat, p_val = stats.ttest_ind(group_A, group_B, equal_var=False)
10     print(f"mean A = {group_A.mean():.2f}, mean B = {group_B.mean():.2f}")
11     print(f"t = {t_stat:.3f}, p = {p_val:.4f}")
12
13     # Effect size: Cohen's d
14     pooled_sd = np.sqrt((group_A.var(ddof=1) + group_B.var(ddof=1)) / 2)
15     cohens_d = (group_A.mean() - group_B.mean()) / pooled_sd
16     print(f"Cohen's d = {cohens_d:.3f} (>= 0.2 small, 0.5 medium, 0.8 large)")

```

Listing 4.2: Welch's two-sample t-test for unequal variances

4.7 Paired t-Test

Question: Do paired measurements differ on average? Each observation in group A is naturally linked to one in group B (same subject, before/after, two algorithms on the same dataset).

$$H_0 : \mu_d = 0, \quad d_i = x_i - y_i, \quad t = \frac{\bar{d}}{s_d/\sqrt{n}}.$$

ML Application – Model Comparison

Comparing two models on the *same* k cross-validation folds: each fold gives one accuracy for model A and one for model B. The folds make natural pairs, so a paired t-test is more powerful than the independent one.

```

1      import numpy as np
2      from scipy import stats
3      from sklearn.model_selection import StratifiedKFold
4      from sklearn.linear_model import LogisticRegression
5      from sklearn.ensemble import RandomForestClassifier
6      from sklearn.datasets import load_breast_cancer
7
8      X, y = load_breast_cancer(return_X_y=True)
9      cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)

```

```

10
11     acc_lr, acc_rf = [], []
12     for tr, te in cv.split(X, y):
13         lr = LogisticRegression(max_iter=5000).fit(X[tr], y[tr])
14         rf = RandomForestClassifier(n_estimators=100, random_state=0).fit(X[
15             tr], y[tr])
16         acc_lr.append(lr.score(X[te], y[te]))
17         acc_rf.append(rf.score(X[te], y[te]))
18
19     acc_lr, acc_rf = np.array(acc_lr), np.array(acc_rf)
20     print(f"LR   mean = {acc_lr.mean():.4f}")
21     print(f"RF   mean = {acc_rf.mean():.4f}")
22
23     t_stat, p_val = stats.ttest_rel(acc_lr, acc_rf)
24     print(f"Paired t-test  t = {t_stat:.3f}, p = {p_val:.4f}")

```

Listing 4.3: Paired t-test to compare two classifiers on CV folds

4.8 Two-Proportion z-Test (A/B Testing)

Question: Do two groups have different conversion rates?

$$H_0 : p_A = p_B, \quad z = \frac{\hat{p}_A - \hat{p}_B}{\sqrt{\hat{p}(1 - \hat{p}) \left(\frac{1}{n_A} + \frac{1}{n_B} \right)}}$$

where $\hat{p} = (x_A + x_B)/(n_A + n_B)$ is the pooled proportion.

A/B Test on Click-Through Rate

Variant A: 1,200 conversions out of 10,000 visitors.

Variant B: 1,320 conversions out of 10,000 visitors.

Is the lift real or noise?

```

1  import numpy as np
2  from statsmodels.stats.proportion import proportions_ztest,
3      proportion_confint
4
5  successes = np.array([1200, 1320]) # conversions
6  trials    = np.array([10_000, 10_000])
7
8  z, p_val = proportions_ztest(successes, trials)
9  print(f"p_A = {successes[0]/trials[0]:.4f}")
10 print(f"p_B = {successes[1]/trials[1]:.4f}")
11 print(f"z = {z:.3f}, p-value = {p_val:.4f}")
12
13 # 95% CI for the difference (Wilson)
14 ci_A = proportion_confint(successes[0], trials[0], method="wilson")
15 ci_B = proportion_confint(successes[1], trials[1], method="wilson")
16 print(f"95% CI A: {ci_A}, 95% CI B: {ci_B}")

```

Listing 4.4: Two-proportion z-test for an A/B experiment

4.9 Chi-Squared Test

4.9.1 Test of Independence

Question: Are two categorical variables independent?

$$\chi^2 = \sum \frac{(O - E)^2}{E},$$

where O = observed count, E = expected count under independence.

ML Application – Categorical Feature Selection

For a classification task, the chi-squared statistic between each categorical feature and the target ranks features by how much they “disagree” with independence. `sklearn.feature_selection.chi2` computes exactly this.

```

1  import numpy as np
2  from scipy.stats import chi2_contingency
3
4  # Rows: device type; Columns: clicked / not clicked
5  observed = np.array([
6  [120, 80],      # mobile
7  [ 60, 140],     # desktop
8  [ 30, 90],      # tablet
9  ])
10
11  chi2, p, dof, expected = chi2_contingency(observed)
12  print(f"chi2 = {chi2:.3f}, p = {p:.4f}, dof = {dof}")
13  print("Expected (under independence):\n", expected.round(1))

```

Listing 4.5: Chi-squared independence test on a contingency table

```

1  from sklearn.feature_selection import SelectKBest, chi2
2  from sklearn.datasets import load_iris
3  import numpy as np
4
5  X, y = load_iris(return_X_y=True)
6  X = (X * 10).astype(int)          # chi2 needs non-negative ints/
   counts
7
8  selector = SelectKBest(chi2, k=2).fit(X, y)
9  print("Chi2 scores per feature:", selector.scores_.round(2))
10 print("Top-2 feature indices  :", np.argsort(selector.scores_) [-2:])

```

Listing 4.6: Chi-squared feature selection in scikit-learn

4.10 ANOVA (F-test)

Question: Do $k \geq 3$ groups all have the same mean?

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_k, \quad F = \frac{\text{between-group variance}}{\text{within-group variance}}.$$

ML Application – Continuous Feature Selection

`sklearn.feature_selection.f_classif` runs an ANOVA F-test of each continuous feature against the class label. It’s the default univariate filter for classification with continuous inputs.

```

1  import numpy as np
2  from scipy import stats
3
4  np.random.seed(0)

```

```

5     acc_v1 = np.random.normal(0.84, 0.01, 30)
6     acc_v2 = np.random.normal(0.86, 0.01, 30)
7     acc_v3 = np.random.normal(0.85, 0.01, 30)
8
9     F, p = stats.f_oneway(acc_v1, acc_v2, acc_v3)
10    print(f"F = {F:.3f}, p = {p:.4f}")
11    if p < 0.05:
12    print("At least one variant differs -- run post-hoc tests (Tukey HSD)
        .")

```

Listing 4.7: One-way ANOVA across three model variants

```

1     from sklearn.feature_selection import SelectKBest, f_classif
2     from sklearn.datasets import load_iris
3
4     X, y = load_iris(return_X_y=True)
5     selector = SelectKBest(f_classif, k="all").fit(X, y)
6     print("F-scores:", selector.scores_.round(2))
7     print("p-values:", selector.pvalues_.round(6))

```

Listing 4.8: ANOVA-based feature ranking

4.11 Non-Parametric Tests

When normality is suspect or sample sizes are tiny, switch to rank-based tests.

```

1     import numpy as np
2     from scipy import stats
3
4     # Heavily skewed (non-normal) data
5     np.random.seed(0)
6     g1 = np.random.exponential(scale=1.0, size=40)
7     g2 = np.random.exponential(scale=1.3, size=40)
8
9     # Mann-Whitney U: non-parametric two-sample location test
10    u, p_u = stats.mannwhitneyu(g1, g2, alternative="two-sided")
11    print(f"Mann-Whitney U = {u:.1f}, p = {p_u:.4f}")
12
13    # Wilcoxon signed-rank: paired non-parametric test
14    paired_a = np.random.exponential(1.0, 30)
15    paired_b = paired_a + np.random.normal(0.1, 0.3, 30)
16    w, p_w = stats.wilcoxon(paired_a, paired_b)
17    print(f"Wilcoxon W = {w:.1f}, p = {p_w:.4f}")

```

Listing 4.9: Mann–Whitney U and Wilcoxon signed-rank

4.12 Kolmogorov–Smirnov Test (and Drift Detection)

The KS test compares two empirical CDFs:

$$D_{n,m} = \sup_x |F_{1,n}(x) - F_{2,m}(x)|.$$

ML Application – Data Drift Detection

In production, run a KS test on each input feature: training distribution vs. live distribution. A significant p-value flags drift – a leading indicator that the model’s accuracy is about to degrade.


```
1      import numpy as np
2      from scipy import stats
3
4      # Training distribution
5      np.random.seed(0)
6      train_feature = np.random.normal(0, 1, 5_000)
7
8      # Live data slowly shifting
9      live_feature = np.random.normal(0.4, 1.1, 1_000)
10
11     ks_stat, p = stats.ks_2samp(train_feature, live_feature)
12     print(f"KS statistic = {ks_stat:.3f}, p = {p:.4g}")
13     if p < 0.01:
14         print("ALERT: distribution drift detected on this feature.")
```

Listing 4.10: KS test for distribution drift between training and live data

4.13 Test for Linear Correlation

Question: Is the population Pearson correlation $\rho \neq 0$?

```
1      import numpy as np
2      from scipy import stats
3
4      np.random.seed(0)
5      x = np.random.normal(0, 1, 200)
6      y = 0.4 * x + np.random.normal(0, 1, 200)
7
8      r, p = stats.pearsonr(x, y)
9      print(f"Pearson r = {r:.3f}, p = {p:.4f}")
10
11     rho, p_s = stats.spearmanr(x, y)
12     print(f"Spearman rho = {rho:.3f}, p = {p_s:.4f}")
```

Listing 4.11: Pearson and Spearman correlation tests